

Monika

Agent が必要とする情報を、参照・検査・更新するための CLI 基盤

現状の課題

LLM にはコンテキストウィンドウがある

大きめのソフトウェア開発では、通常の実装作業でもコンテキストが尽きる

最初から最後まで、全ての設計・コード・ログ・判断を入れ続けることは難しい

いわゆる長期記憶に類するものは、モデルの外で補う必要がある

解決策とされているもの

ハーネスやモデルプロバイダによる圧縮・要約
MEMORY.md のような単一ファイルに記憶を書く方式
コード内コメントに経緯や注意点を書き込む方式
文書として設計や判断を保存する方式

それぞれの弱いところ

圧縮・要約だけでは、重要な情報を全て保持して必要時に取り出すことは難しい

単一の記憶ファイルは、多くの情報を書きにくく、あとで探索しにくい

コード内コメントは、設計、実験結果の解釈、運用上の判断などを置く場所としては局所的すぎる

文書はたくさん保存できるが、コードやデータとの結びつきが弱くなりやすい

文書での記録

いくらでも保存できる

フォルダ分けができる

実装の意図、経緯、ベンチマークからの判断などを書ける

ただし、データやコードとの結びつきが弱い

データやコードがないと、知識として十分に使いにくい

文書単体は弱い

現代のプログラミング言語は、単なる文書にはない抽象化や安全機構を持っている

モジュールシステム、型システム、自動 API ドキュメント、テスト、ビルドなど

複雑なソフトウェアでは、実装して制約が分かってから仕様を更新することも多い

そのため、文書だけを正としてコードを一方向に作る運用では足りない

コード、文書、データなどを同時に変更し、互いに結びつけたい

今回の提案

Agent が必要とする情報を、形式をまたいで扱うための抽象化を作る

対象は Markdown だけではなく、ソースコード、JSONL、ログ、
Web 由来の内容、未知形式のファイルなど
ファイル全体だけでなく、段落、関数、型、実験データの一行、ログ
の一部分も扱う
それらを Agent が使う CLI として提供する

なぜプロジェクトレベルで扱うか

Markdown に「このファイルの何行目」と書くだけでは足りない
コードの型定義や関数は、名前変更やフォーマットで行番号が変わる
JSONL、DB、ログ、Web 由来の内容などは、行番号だけでは表しにくい

毎回ばらばらに小さな管理ツールを作ると、参照の検査や更新をまとめて扱えない

参照方法、検査方法、更新方法のある程度統一して扱える形式と CLI が必要になる

Monika の中核モデル

Resource: 観測しようとする対象

Observation: ある時点で得られた固定の観測結果

Artifact: 現在の CLI で扱う content-backed な観測結果

Region: 観測結果の全体または部分領域

Reference: Region を指すための値

Annotation / Relation: Region に付与される情報や関係

Region と selector

Region は単なる行番号ではない

Markdown では段落や見出しを選べる

Source code では関数、型、symbol などを選べるようにする

JSONL では条件に一致する行を選べる

未知形式では byte range や拡張 selector を使える

selector を interpreter が解釈し、解決できない場合は明示的に失敗として扱う

Reference を文字列だけにしない

Reference は path や URL の文字列ではなく、target、selector、binding、expectation を持つ値

解決結果は content identity と観測時刻を持つ snapshot として残せる

以前見た対象から変わっていないか、selector が古くなっていないかを検査できる

解決できない selector を、似ている別の region に自動でずらさない

Annotation の置き場所

同じ annotation は、複数の場所に表現できる

Markdown inline annotation

source comment

sidecar file

generated index

Monika は annotation の意味と、どこに書かれていたかを分けて保持する

inline-only、sidecar-only、divergent などを検出できる

書き込みは patch 経由

interpreter や deriver はワークスペースを直接書き換えない
変更が必要な場合は ProposedPatch を返す
patch には、理由、由来、変更前後の content identity、具体的な
edit が含まれる
apply が、対象が変わっていないことと、結果の content identity
を検証して書き込む
同じ patch を再適用した場合は no-change になる

derive と infer を分ける

derive は、すでに明示されている情報から別の明示表現を導く処理
例: Markdown inline annotation から sidecar entry を生成する
同じ入力から同じ patch を返す
apply 後にもう一度 derive しても、不要な差分を出さない
infer は、明示されていない関係を推測する処理
現在の中核機能では infer を行わない

CLI の分担

scan: workspace 内の artifact を列挙する

inspect: artifact から region、reference、annotation を抽出する

resolve: reference を解決し、snapshot を返す

check: 壊れた参照や不整合を診断する

derive: 明示情報から patch を提案する

apply: patch を検証して適用する

capabilities / extension test: 組み込み機能と拡張の契約を確認する

Agent からの使い方

まず related で関係する artifact を絞る

必要な artifact だけ read する

厳密な機械処理が必要な場合は inspect や related -json を使う
workspace 全体の参照や annotation の不整合は check で調べる
明示情報の同期は derive で patch を作り、apply で検証して適用する

現在の実装範囲

OCaml の参照実装 Sugar が意味モデルと正規形を持つ
Rust の高速実装 Bitter は、現時点では一部の parity slice から進めている
schema、diagnostic code、CLI contract、golden test を仕様層として固定している
Markdown と sidecar の inspect、basic check、inline-to-sidecar derive、apply の最初の範囲が動いている
extension は明示指定した一時 interpreter に対する describe、observe、同一 session の resolveRegion まで実装している

拡張していく対象

Web、PDF、Rust、Python、Parquet、実験基盤、検索 index、
変更影響解析などを後から足せるようにする
追加するものは、Resource を観測し、Observation から Region
を解決する capability として扱う
extension は言語非依存の値と操作で接続する
core は、selector、reference、diagnostic、patch、snapshot
の共通部分を維持する

まとめ

Agent の長期的な作業では、情報をモデル外に持つ必要がある
文書は必要だが、コードやデータとの結びつきを機械的に扱える必要がある

Monika は artifact、region、reference、annotation、patch を
共通の値として扱う

それにより、必要な情報だけを読み、不整合を検査し、安全に同期するための CLI 境界を提供する